Unified Trading Platform — Engineering Intake Report

Prepared by: Staff-Level Engineering Review
Codebase: unified-trading-platform
Review Date: 2026-02-26
Status: Paper Trading Only (Alpaca paper endpoint hardcoded)

---

1. Architecture Summary

What This System Does

This is a fully autonomous algorithmic trading platform that combines a streaming ML inference pipeline ("AWET") with QuantConnect Lean-style technical alpha signals. It ingests live market data, computes features, runs a Temporal Fusion Transformer (TFT) ML model for quantile price predictions, blends those predictions with RSI/EMA/MACD alpha signals, passes the ensemble through a risk engine, and submits paper trades to Alpaca.

The platform is currently **paper-trading only** — the Alpaca client hard-blocks any non-paper endpoint, and there are three layered execution safety gates.

Stack

Layer	Technology
Language	Python 3.11+
ML Model	PyTorch TFT → exported to ONNX for inference
Message Bus	Apache Kafka (Confluent CP 7.6) with Avro schemas
Database	TimescaleDB (PostgreSQL 15 extension)
State Cache	Redis (kill switch + risk state persistence)
Observability	Prometheus + Grafana (11 alert rules)
Experiment Tracking	MLflow
Orchestration	Docker Compose (single-host)
Broker	Alpaca Paper Trading API (httpx async)
Data Sources	Polygon.io, yfinance, Reddit (pushshift ZST)

Service Architecture

```
[Polygon / yfinance / Alpaca]
        │
        ▼
 DataIngestion (8001) ──Kafka[market.raw]──► FeatureEngineering (8002)
                                  │
                            Kafka[market.engineered]
                                  │
                                  ▼
                           ModelInference / TFT (8003)
                                  │
                            Kafka[predictions.tft]
                                  │
 LeanAlpha (8014) ──Kafka[alpha.signals]──►  SignalEngine (8015)
 (RSI / EMA / MACD)                     │
                            Kafka[signals.scored]
                                  │
                                  ▼
                           RiskEngine (8004)
                                  │
                            Kafka[risk.approved]
                                  │
                                  ▼
                           ExecutionAgent (8005)
                           ├── AlpacaClient (paper)
                           └── TradesRepository (TimescaleDB)
                                  │
                                  ▼
                           ExitMonitor (8010)
                           (TP / SL / time exit)

Supporting:
  - Backtester (8012)        — offline walk-forward validation
  - ModelTraining            — TFT train + ONNX export + MLflow
  - MonitoringAgent (8013)   — health reports
  - Orchestration scripts    — 40+ ops/admin scripts
```

Entry Points

File	Purpose
`infra/docker-compose.yml`	Primary startup; all services + infra

File	Purpose
`services/execution/main.py`	`ExecutionAgent.run()` — critical path end
`services/risk_engine/main.py`	`RiskEngineService.run()`
`services/signal_engine/main.py`	`SignalEngineService.run()`
`services/lean_alpha/main.py`	`LeanAlphaService.run()`
`services/data_ingestion/main.py`	`DataIngestionAgent.run()`
`infra/db/init.sql`	Database schema bootstrap
`configs/app.yaml`	Primary config (safety gates, sizing, throttles)
`configs/limits.yaml`	Risk limits (position size, daily loss, kill switch)

2. Module-by-Module Map

Module	Purpose	Key Files	Dependencies	Risk Level
`services/data_ingestion`	Ingest OHLCV bars from Polygon / yfinance; publish to `market.raw`	`main.py`, `providers.py`, `backfill/`	Polygon API, yfinance, Kafka, TimescaleDB	Medium
`services/feature_engineering`	Compute returns, volatility, RSI, MACD, Bollinger Bands from raw bars	`engine.py`, `main.py`	TimescaleDB, Kafka	Medium
`services/lean_alpha`	Run RSI-14, EMA-cross (12/26), MACD (12/26/9) on live price stream	`main.py`, `rsi_alpha.py`, `ema_cross_alpha.py`, `macd_alpha.py`	Kafka	Medium

Module	Purpose	Key Files	Dependencies	Risk Level
services/signal_engine	Weighted ensemble: TFT (0.5) + RSI (0.2) + EMA (0.2) + MACD (0.1) → signals.scored	main.py, ensemble.py, filters.py, regime.py	Kafka, TimescaleDB	**HIGH**
services/model_inference	Load ONNX TFT model; predict quantile returns (q10/q50/q90); publish to predictions.tft	engine.py, main.py	ONNX Runtime, Kafka	**HIGH**
services/model_training	Train TFT; walk-forward validation; export to ONNX; register in MLflow	train.py, train_tft.py, walk_forward.py, onnx_engine.py, registry.py	PyTorch, MLflow, TimescaleDB	Medium
services/risk_engine	Evaluate position size, daily loss, volatility, CVaR; kill switch; Redis state	engine.py, main.py, portfolio_risk.py	Redis, TimescaleDB	**CRITICAL**
services/execution	Final execution gate; idempotency; throttles; exposure caps; Alpaca bracket orders	main.py, integrations/alpaca_client.py, integrations/trades_repository.py	Alpaca Paper API, TimescaleDB, Kafka	**CRITICAL**
services/exit_monitor	Monitor open positions; auto-exit on TP/SL/time	main.py	Alpaca Paper API, TimescaleDB	**HIGH**
services/backtester	Offline simulation on historical features	main.py, metrics.py	TimescaleDB	Low
services/monitoring	Health checks, Prometheus metrics, Grafana dashboards	main.py, health_report.py	Prometheus	Low

Module	Purpose	Key Files	Dependencies	Risk Level
`shared/schemas`	Avro schema definitions for all Kafka topics	`*.avsc`	—	Medium
`shared/db`	DB connection utilities	`connection.py`	asyncpg	Medium
`infra/`	Docker Compose, TimescaleDB init SQL, Grafana/Prometheus config	`docker-compose.yml`, `db/init.sql`	—	**HIGH**
`configs/`	All runtime configuration	`app.yaml`, `limits.yaml`, `kafka.yaml`	—	**HIGH**
`tests/`	Unit + integration + regression test suites	`unit/`, `integration/`	pytest	Medium

---

3. Risk Report

3.1 Security Risks

[CRITICAL] Default database credentials committed to config
Found in `services/signal_engine/main.py:31`, `services/feature_engineering/main.py:69`, `services/execution/integrations/trades_repository.py:628`, `services/exit_monitor/main.py:113`, and 6+ other files. Every service falls back to `POSTGRES_PASSWORD='awet'` if the env var is absent. If these services ever run without proper env injection (e.g., a local `docker compose up` without `.env`), the DB is accessible with trivially guessable credentials.
Fix: Remove all defaults from code. Require the env var to be set; raise an error at startup if missing.

[HIGH] DSN construction with f-strings logs partial credentials
`trades_repository.py` logs `dsn=self.dsn[:30] + "..."` — the first 30 characters of a PostgreSQL DSN always include the username and may include part of the password.
Fix: Log only `host:port/dbname`, never reconstruct the DSN in a log line.

[MEDIUM] No secrets scanning / pre-commit hooks in repo
No `.gitignore` entry for `.env`, no `detect-secrets` or `truffleHog` in CI/CD (no CI config found at all).
Fix: Add `detect-secrets` pre-commit hook; ensure `.env` is in `.gitignore`.

[MEDIUM] Dashboard server uses default password as a fallback for direct DB access
`services/orchestration/scripts/dashboard_server.py` makes asyncpg connections with the same default credential pattern.

3.2 Trading Risks

[CRITICAL – BUG] `PositionState` has duplicate `to_dict` and `from_dict` methods
In `services/risk_engine/engine.py`, `PositionState` defines `to_dict()` at line 64 and again at line 101, and `from_dict()` at line 112 and again at line 138. Python silently uses the **last** definition. The first `to_dict` (which serializes correctly) is dead code; the second one happens to be identical — but this is fragile and a maintenance landmine. More critically, if the two definitions ever diverge (e.g., a developer adds a field to one but not the other), Redis state will silently persist/restore corrupt data. The current dual definitions indicate this was copy-pasted without cleanup.
Fix: Delete the first pair of definitions (lines 64–126). Keep only one canonical `to_dict`/`from_dict`.

[CRITICAL – BUG] Risk limits YAML path mismatch — limits never load from file
`RiskEngine._load_config()` constructs the config path as:
`Path(__file__).parent.parent.parent / "config" / "limits.yaml"`
The actual file is at `configs/limits.yaml` (plural). The directory `config/` does not exist. As a result, `config_path.exists()` always returns `False`, and all risk limits silently fall back to Python defaults. The per-symbol NVDA/TSLA overrides in `limits.yaml` are **never applied**.
Fix: Change `"config"` to `"configs"` in the path construction, or better, read the path from an environment variable `RISK_LIMITS_PATH`.

[CRITICAL] Kafka `enable.auto.commit: True` in signal engine and lean alpha
`services/signal_engine/main.py:175` and `services/lean_alpha/main.py:123` both use `enable.auto.commit: True`. This means Kafka offsets are committed on a timer, independent of whether the message was successfully processed. If the process crashes between offset commit and database write (or order submission), those messages are silently lost — no signal reaches the risk engine for those bars.
Fix: Disable auto-commit. Use manual `consumer.commit()` after successful processing, matching the pattern already used correctly in the `ExecutionAgent`.

[HIGH] No market hours gate in execution path
`ExecutionAgent._process_message()` submits orders to Alpaca without checking whether the market is open. Alpaca will reject orders submitted outside regular market hours (9:30am–4:00pm ET) for `time_in_force: day` orders, returning a rejection that gets logged as `alpaca_order_rejected` but does not halt downstream processing. The trade is recorded as `blocked` due to the rejection, but the signal is consumed and lost — not re-queued for market open.
Fix: Call `AlpacaClient.get_clock()` at message processing time (or cache it with a 30-second TTL). Block execution if `is_open = False`. Optionally, hold the message for re-processing at open.

[HIGH] Signal staleness — no max-age check on TFT signal before ensemble compute
`SignalEnsemble.compute()` uses whatever `TftSignal` is stored in `self._tft[symbol]` with no TTL check. Alpha

signals correctly expire via `SIGNAL_TTL_SECONDS` (5 minutes), but a TFT signal could be hours old if model inference falls behind. The ensemble would blend a fresh alpha signal with a stale ML prediction and emit a confident directional signal.
Fix: Add a `TFT_SIGNAL_MAX_AGE_SECONDS` check in `compute()`:

```python
if (datetime.now(timezone.utc) - tft.ts).total_seconds() > TFT_SIGNAL_MAX_AGE_SECONDS:
    logger.warning("stale_tft_signal", symbol=symbol, age_seconds=age)
    return None
```

[HIGH] Risk engine state is a mutable singleton without locks
`get_risk_engine()` returns a module-level singleton `_engine`. `RiskEngine.evaluate()` reads and `update_pnl()` writes to `self.state` (a mutable `PositionState` dataclass) without any lock. In an async event loop, two concurrent Kafka messages for different symbols could race on `state.daily_pnl`, producing incorrect P&L tracking and potentially failing to trigger the kill switch at the correct threshold.
Fix: Wrap state mutations in `asyncio.Lock()`. Or better, move state management fully to Redis (which is already plumbed in) and use Redis atomic operations.

[MEDIUM] Idempotency key for signals is timestamp-based, not content-based
`signal_engine/main.py` constructs `idempotency_key = f"signal:{symbol}:{result.ts.isoformat()}"`. If the signal engine restarts and reprocesses the same prediction message (due to Kafka offset reset), it will compute the same `result.ts` and the `ON CONFLICT DO NOTHING` in the DB insert will correctly deduplicate the DB row — but the Kafka `produce` call upstream has no deduplication. A second copy of the signal can propagate to the risk engine and generate a duplicate trade attempt. The execution agent does have idempotency checking, so this is caught downstream, but it creates unnecessary noise.
Fix: Pass the upstream prediction's `event_id` (UUID) through as the signal's idempotency key rather than deriving it from the timestamp.

[MEDIUM] Naive `datetime.now()` calls in orchestration scripts
`services/orchestration/scripts/run_pipeline.py`, `run_backtest.py`, and `daily_pnl_report.py` use `datetime.now()` (timezone-naive). When these are compared with timezone-aware timestamps from the DB (which stores `TIMESTAMPTZ`), Python will raise `TypeError: can't compare offset-naive and offset-aware datetimes` — but only at runtime when a comparison is attempted, meaning this is a latent bug that surfaces under specific code paths.
Fix: Replace all `datetime.now()` with `datetime.now(timezone.utc)` across orchestration scripts.

[MEDIUM] No daily P&L reset scheduler
`RiskEngine.reset_daily()` exists but there is no scheduled call to invoke it at market open. If the process runs across midnight or across multiple trading days, `state.daily_pnl` accumulates indefinitely. This would make the daily loss check increasingly conservative over time — eventually blocking all trades — or, if the kill switch fires based on multi-day accumulated loss, produce incorrect results.
Fix: Add a scheduled coroutine that calls `risk_engine.reset_daily()` at 9:30am ET on each trading day.

[LOW] Hard-coded local data path in committed config
`configs/app.yaml` has `data.data_root: "/home/kironix/train"` — a machine-specific absolute path committed to source control. Any developer or CI environment that clones this repo and runs data scripts will get file-not-found errors.
Fix: Replace with an environment variable: `data_root: ${DATA_ROOT:-/data/train}`.

---

4. Refactor + Roadmap Plan

2-Week Sprint Plan

Day 1–2 (Sprint 1): Critical Bug Fixes — Zero Trading Safety

Task 1.1 — Fix PositionState duplicate methods
Goal: Eliminate the dual `to_dict`/`from_dict` bug before it causes silent data corruption.
Files: `services/risk_engine/engine.py`
Acceptance Criteria: Only one `to_dict` and one `from_dict` exist on `PositionState`. Existing Redis serialization round-trip test passes.
Test Plan: `pytest tests/unit/test_risk_engine.py -k state` + add round-trip test: serialize → deserialize → verify field equality.

Task 1.2 — Fix risk limits YAML path (`config` → `configs`)
Goal: Ensure per-symbol risk overrides (NVDA, TSLA) actually load.
Files: `services/risk_engine/engine.py:_load_config()`
Acceptance Criteria: Add a startup log assertion. `RiskEngine._load_config()` logs `risk_limits_loaded` with `source="configs/limits.yaml"`. NVDA max_position_pct reads as 0.015.
Test Plan: Add unit test that instantiates `RiskEngine()` and asserts `engine.get_limits_for_symbol("NVDA").max_position_pct == 0.015`.

Day 3–4 (Sprint 2): Kafka Safety — Manual Commit

Task 2.1 — Disable Kafka auto-commit in SignalEngine and LeanAlpha
Goal: Prevent silent message loss on crash between poll and DB/produce.
Files: `services/signal_engine/main.py`, `services/lean_alpha/main.py`
Changes:

- Set `"enable.auto.commit": False` in consumer config
- Add `consumer.commit()` after each successful `_process_all_symbols()` / alpha publish
- Wrap in try/except; only commit on success; log and continue on error (don't commit)

Acceptance Criteria: If process is killed mid-loop, on restart the message is reprocessed. No duplicate DB inserts (idempotency prevents them).
Test Plan: `pytest tests/unit/test_live_ingestion.py` + `tests/integration/test_replay_determinism.py`.

Day 5–6 (Sprint 3): Credential and Config Hygiene

Task 3.1 — Remove default database credentials from service code

Goal: Fail fast on misconfigured deployments rather than silently using `'awet'` password.
Files: `services/signal_engine/main.py`, `services/feature_engineering/main.py`, `services/exit_monitor/main.py`, `services/execution/integrations/trades_repository.py`, and 6+ orchestration scripts.
Change pattern:

```python
# Before
os.getenv('POSTGRES_PASSWORD', 'awet')

# After
password = os.getenv('POSTGRES_PASSWORD')
if not password:
    raise EnvironmentError("POSTGRES_PASSWORD env var is required")
```

Acceptance Criteria: Starting any service without `POSTGRES_PASSWORD` set raises a clear error at startup, not a connection error at first query.
Test Plan: Run `POSTGRES_PASSWORD= python -m services.signal_engine.main` → expect `EnvironmentError`. Add to `tests/unit/test_structured_logging.py`.

Task 3.2 — Replace hard-coded data path in app.yaml

Files: `configs/app.yaml`, all scripts that reference `data_root`
Change: `data_root: "${DATA_ROOT:-/data/train}"`
Test Plan: `configs/app.yaml` passes a YAML lint + schema check. `smoke_ingest.py` uses `$DATA_ROOT` from env.

Day 7–8 (Sprint 4): Signal Staleness + Market Hours Gate

Task 4.1 — Add TFT signal TTL to SignalEnsemble

Files: `services/signal_engine/ensemble.py`
Change: Add `TFT_SIGNAL_MAX_AGE_SECONDS = int(os.getenv("TFT_SIGNAL_MAX_AGE_SECONDS", "600"))` (10 minutes default). In `compute()`, return `None` if TFT signal age exceeds limit. Log `stale_tft_signal` with symbol and age.
Acceptance Criteria: `compute()` returns `None` for a 15-minute-old TFT signal. Test: mock `datetime.now` to

advance time.
Test Plan: `tests/unit/test_lean_alpha_and_signal_engine.py`.

Task 4.2 — Add market hours gate to ExecutionAgent
Files: `services/execution/main.py`, `services/execution/integrations/alpaca_client.py`
Change: Cache market clock (`is_open`, `next_open`) with 60-second TTL. In `_process_message`, if `is_open == False`, block with `error_message = "market_closed"` and route to `execution.blocked`. Log `market_closed_block` with `next_open`.
Acceptance Criteria: No Alpaca order submission when market is closed. Trade is recorded as blocked with `error_message="market_closed"`.
Test Plan: Mock `get_clock()` to return `is_open=False`. Assert `OrderResult` is not called. `tests/unit/test_alpaca_client.py` + `tests/integration/test_execution_safety_gates.py`.

Day 9–10 (Sprint 5): Risk Engine Thread Safety + Daily Reset

Task 5.1 — Add asyncio.Lock to RiskEngine state mutations
Files: `services/risk_engine/engine.py`
Change: Add `self._lock = asyncio.Lock()` in `__init__`. Wrap `evaluate()`, `update_pnl()`, `activate_kill_switch()`, `reset_daily()` with `async with self._lock`.
Acceptance Criteria: Concurrent evaluation of 10 symbols does not produce race conditions on `daily_pnl`. Existing `test_risk_engine.py` passes. New concurrency test: fire 20 concurrent `evaluate()` calls and verify final `daily_pnl` is consistent.

Task 5.2 — Add market-open daily reset scheduler
Files: `services/risk_engine/main.py`
Change: Add a background `asyncio` task that sleeps until 9:30am ET each day and calls `risk_engine.reset_daily()`. Use `zoneinfo.ZoneInfo("America/New_York")` (stdlib, Python 3.9+).
Acceptance Criteria: `reset_daily()` is called exactly once per trading day at market open. Log entry `daily_risk_reset` appears in structured logs.
Test Plan: Unit test with mocked `datetime.now` advancing past 9:30am ET.

Day 11–12 (Sprint 6): Observability Hardening

Task 6.1 — Add Prometheus alert for stale TFT signal
Files: `infra/prometheus/alerts.yml`
Change: Add alert `TFTSignalStaleness` that fires when `signal_engine_last_tft_signal_age_seconds > 600`. Add corresponding Prometheus counter to `SignalEngineService`.

Task 6.2 — Add structured log for risk config load failure
Files: `services/risk_engine/engine.py`
Change: On YAML load failure, log at `ERROR` level (currently `WARNING`) with the full exception. Add Prometheus counter `risk_config_load_errors_total`.

Task 6.3 — Timezone audit of orchestration scripts
Files: All `services/orchestration/scripts/*.py`
Change: Global replace `datetime.now()` → `datetime.now(timezone.utc)`. Run `grep -rn "datetime.now()" services/orchestration` and fix all instances.
Test Plan: `tests/unit/test_pnl_report.py` + `tests/unit/test_reconciliation.py`.

Day 13–14 (Sprint 7): Test Coverage and CI Scaffolding

Task 7.1 — Add GitHub Actions (or equivalent) CI config
Files: `.github/workflows/ci.yml` (new)
Contents: `pytest tests/unit/` on every PR. Docker Compose smoke test on merge to main. `detect-secrets` scan on every push.

Task 7.2 — Fill critical test gaps
Files: `tests/unit/test_risk_engine.py`, `tests/unit/test_lean_alpha_and_signal_engine.py`, `tests/integration/test_execution_safety_gates.py`
Priority test cases:

- Risk limits YAML load round-trip (Task 1.2 regression)
- PositionState serialize/deserialize round-trip with all fields
- Ensemble returns `None` for stale TFT signal
- ExecutionAgent blocks when market is closed
- Concurrent `RiskEngine.evaluate()` does not race on `daily_pnl`

Big Refactors (Only If Necessary, Post-Sprint)

Centralize DSN construction. All 10+ services build the DB DSN from env vars inline. Extract a single `shared/db/dsn.py` module with one `build_dsn()` function that validates all required vars at import time.

Move risk state to Redis exclusively. The in-memory `PositionState` is a source of truth only while the process is alive; Redis is the durable store. Refactor so the in-process state is always written-through to Redis, and on startup the state is *always* restored from Redis (not just when Redis is available). This eliminates the lock concern entirely.

Replace the orchestration script sprawl with a proper CLI. There are 40+ loose Python scripts in `services/orchestration/scripts/`. These should be grouped into a structured CLI package (e.g., using `click` or `typer`) with shared config loading, logging setup, and error handling.

---

5. Questions for You

1. **Live trading timeline:** Is live trading (non-paper) being considered? The current paper-only guardrails are solid, but several changes (e.g., removing the `ALLOWED_BASE_URLS` constraint) would require explicit review. What's the target date, if any?
2. **Risk state source of truth:** If the risk engine process crashes and Redis is unavailable, it starts with a clean `PositionState` (zero daily P&L, no active positions). Is this the intended fallback, or should the system halt until Redis is reachable?
3. **Kafka partition strategy:** All topics appear to use the symbol as the partition key, but I couldn't confirm partition counts. How many symbols are in the active universe at peak? This affects whether single-partition topics become a bottleneck.
4. **Model versioning and rollback:** MLflow is wired in, but I don't see an automated rollback trigger. If a newly promoted model produces a string of losing signals, what's the manual or automated path to revert to the previous ONNX checkpoint?
5. **Exit monitor vs. bracket orders:** The system has both `ExitAgent` (polling-based TP/SL/time exit) and `use_bracket_orders: true` (broker-side TP/SL). Are these intended to run simultaneously? If so, there's a risk of double-exit attempts on the same position.
6. **Reddit sentiment integration:** The `data_ingestion/backfill/reddit_loader.py` and `ingest_reddit_*.py` scripts suggest Reddit posts are being ingested — but I don't see Reddit sentiment as an input feature to the TFT model or as an alpha signal. Is this a future feature, or is it already integrated somewhere I missed?
7. **Walk-forward validation gate:** `walk_forward.py` exists and the README mentions a minimum OOS Sharpe of 0.5. Is this gate enforced automatically on the `promote_model.py` path, or is it manual?
8. **Deployment environment:** Docker Compose suggests single-host. Is this running on a single VM/server? Are there plans for HA or multi-host deployment? This affects whether the Redis kill switch and in-memory risk state are reliable.
9. **Timezone of data vs. system:** The Polygon ingestor and TimescaleDB store data in UTC. The market hours gate (to be added) needs to compare against US/Eastern. Is the host server's system timezone relevant anywhere, or is everything UTC-normalized?
10. **CI/CD:** I found no `.github/workflows`, `Jenkinsfile`, or equivalent CI config. Are tests run manually pre-commit, or is there a CI system not in this repo?

Next Step: High-Impact Improvement Implementation

Recommendation: Start with **Task 1.2 (Risk limits YAML path fix)** and **Task 2.1 (Kafka manual commit)**. These are the two bugs that silently corrupt the system's core safety mechanisms — one prevents risk limits from ever loading from file, the other risks message loss in the signal pipeline.

Ready to guide you step-by-step through either. Say **"implement Task 1.2"** or **"implement Task 2.1"** and I'll provide exact code diffs, test cases, and a safe paper-trading validation procedure.